Selección de un framework móvil para ConectaCruz

Rodrigo Ledezma Sanchez

Universidad Privada Domingo Savio

Aplicaciones Móviles I (SIS-0180)

Jimmy Nataniel Requena Llorentty

9 de noviembre de 2026

Selección de un framework móvil para ConectaCruz

Introducción

ConectaCruz se plantea como un catálogo móvil para pequeños negocios de Santa Cruz de la Sierra. El caso parte de un comercio de barrio que recibe consultas y pedidos por WhatsApp y redes sociales. Si sus productos, horarios y disponibilidad se comunican en conversaciones separadas, el cliente puede tener que preguntar lo mismo más de una vez. Esta situación es un supuesto del caso. La actividad no incluyó encuestas, entrevistas ni estimaciones de adopción en Santa Cruz.

La pregunta de investigación fue: ¿qué framework móvil conviene a un emprendimiento local de Santa Cruz que necesita catálogo, pedidos y reservas con un presupuesto limitado? El objetivo fue comparar tres alternativas y comprobar que la opción elegida permitía ejecutar una pantalla inicial de ConectaCruz en Android.

Flutter documenta la reutilización de una base de código entre plataformas (Flutter, s. f.). React Native ofrece guías para Android e iOS y requiere fundamentos de JavaScript (Meta Platforms, 2026). En el enfoque nativo, Kotlin es la opción recomendada por Android Developers (s. f.) y SwiftUI permite crear una aplicación iOS con Swift en Xcode (Apple, s. f.).

Como evidencia académica, Ilovan (2026) comparó implementaciones de una misma aplicación y observó que el rendimiento relativo variaba según la tarea y la plataforma. Ese estudio orienta el análisis de rendimiento, mientras que el presupuesto limitado, la prioridad de Android para la demostración y la experiencia del equipo se tratan como condiciones del caso.

Métodos

Se realizó una comparación documental de Flutter con Dart, React Native con JavaScript y desarrollo nativo con Kotlin para Android y Swift con SwiftUI para iOS. Se consultaron cuatro guías oficiales para identificar capacidades y un estudio empírico para contrastar el rendimiento. Se evaluó el mismo alcance en cada alternativa: una aplicación con catálogo, pedidos y reservas, con Android como plataforma del prototipo e iOS como objetivo posterior.

Criterios y pesos

Costo (30 %). Esfuerzo esperado para implementar y mantener los mismos flujos en ambos sistemas. Se compara esfuerzo relativo, sin cotizaciones en bolivianos.

Rendimiento (20 %). Respuesta esperada al iniciar la aplicación, recorrer una lista y cargar imágenes. La calificación se basa en evidencia publicada y juicio técnico.

Comunidad (15 %). Disponibilidad de documentación, ejemplos y apoyo técnico para resolver problemas. Se valoró la disponibilidad, sin contar usuarios ni repositorios.

Soporte Android e iOS (20 %). Capacidad de entregar una aplicación en ambos sistemas. La duplicación de esfuerzo se contabilizó en costo.

Curva de aprendizaje (15 %). Esfuerzo de incorporación estimado para un equipo que ya trabajó con Flutter en la asignatura.

La escala ordinal fue de 1 a 5 (1 = muy desfavorable, 3 = intermedio y 5 = muy favorable). El aporte se calculó como peso × calificación ÷ 5. La suma de aportes produce un total sobre 100. También se recalculó el resultado suponiendo un equipo con experiencia en React.

Resultados

Tabla 1

Matriz ponderada de alternativas para ConectaCruz

Criterio	Peso (%)	Flutter	React Native	Nativo
Costo	30	5 (30)	5 (30)	2 (12)
Rendimiento	20	4 (16)	4 (16)	5 (20)
Comunidad	15	4 (12)	4 (12)	4 (12)
Soporte Android e iOS	20	5 (20)	5 (20)	5 (20)
Curva de aprendizaje	15	4 (12)	3 (9)	2 (6)
Total ponderado	**100**	**90**	**87**	**70**

Nota. Cada celda muestra la calificación (1–5) y su aporte entre paréntesis. Por ejemplo, 30 × 5 ÷ 5 = 30 puntos en costo.

Flutter obtuvo 90/100, React Native 87/100 y el desarrollo nativo 70/100. La opción propuesta según la matriz fue Flutter. En el escenario alternativo, al cambiar únicamente la curva de aprendizaje de React Native de 3 a 5, su total subió a 93/100.

Verificación del prototipo

La salida guardada de flutter doctor -v registra Flutter 3.47.9, Android SDK 37.0.0, licencias Android aceptadas y el mensaje “No issues found!”. ConectaCruz se ejecutó en el emulador Android Pixel 6 Pro y se capturó la pantalla con el título, una descripción y el botón «Ver negocios». El Apéndice A presenta ambos registros.

Discusión

El costo favoreció por igual a Flutter y React Native por la posibilidad de reutilizar la interfaz y la lógica entre Android e iOS. La diferencia de tres puntos provino de la curva de aprendizaje. En este equipo ya existían un entorno Flutter configurado y una pantalla en Dart, de modo que continuar sobre esa base requería menos incorporación. El empate en soporte reconoce que el enfoque nativo también puede cubrir ambos sistemas mediante implementaciones separadas.

El desarrollo nativo obtuvo el mayor puntaje estimado de rendimiento. Ilovan (2026) reportó menores tiempos agregados en operaciones del cliente ajenas a la interfaz para las versiones nativas, mientras que el orden varió entre tareas y plataformas. Ese resultado no cuantifica la fluidez del catálogo de ConectaCruz. React Native también puede resultar preferible: la matriz lo sitúa en 93/100 si el equipo domina React y conserva el resto de las condiciones.

Las tres opciones obtuvieron 4 en comunidad porque ofrecen guías y recursos de apoyo. La revisión no aportó una medida comparable para distinguirlas en ese criterio. El soporte documentado de iOS se consideró una capacidad de las alternativas, aunque el prototipo se verificó únicamente en Android.

La evaluación no incluyó mediciones de arranque, memoria ni fluidez en equipos físicos, ni una compilación para iOS. Tampoco midió horas de desarrollo o costos de mantenimiento. La ejecución en el emulador verifica el arranque y la presentación de la interfaz en Android. La evidencia muestra el botón de bienvenida, cuya acción estaba pendiente en esa versión. Pedidos y reservas permanecen como alcance futuro.

Para ConectaCruz, se propone continuar en Flutter y validar primero el catálogo con negocios piloto. Antes de ampliar el alcance, se podría medir el arranque y el desplazamiento con imágenes en teléfonos Android de distintas gamas. El paso siguiente sería probar el flujo de pedidos y reservas con estados claros de confirmación, y repetir la validación en iOS cuando el equipo cuente con ese entorno.

Referencias

Android Developers. (s. f.). *Android's Kotlin-first approach*. https://developer.android.com/kotlin/first

Apple. (s. f.). *Hello, SwiftUI*. Apple Developer. https://developer.apple.com/tutorials/develop-in-swift/hello-swiftui

Flutter. (s. f.). *Build for and integrate with multiple platforms*. https://docs.flutter.dev/platform-integration

Ilovan, A. (2026). *Cross-platform vs native mobile development: An empirical study of software quality trade-offs* [Prepublicación]. arXiv. https://doi.org/10.48550/arXiv.2609.21544

Meta Platforms. (2026, 27 de octubre). *Introduction*. React Native. https://reactnative.dev/docs/getting-started

Apéndice A

Evidencia de ejecución en Android

Figura A1

Pantalla de bienvenida en el emulador Android



Nota. Captura de la ejecución del 9 de noviembre de 2026.

Diagnóstico de Flutter

```
Flutter 3.47.9 · Android SDK 37.0.0
All Android licenses accepted.
No issues found!
```

Extracto del registro original flutter_doctor.txt, conservado en evidencias_actividad_01.